

รายงานตรวจสอบความสอดคล้อง: วิทยานิพนธ์ ↔ Source Code

โครงการ InsightReview — เว็บแอปเปลี่ยนรีวิวร้านอาหารจาก Google Maps ให้กลายเป็นกลยุทธ์ทางธุรกิจ | มุมมอง: Research Reviewer / Software Engineer / NLP-ML Engineer / System Analyst / Thesis Examiner | วันที่ตรวจ: 12 มิ.ย. 2569

1. บทสรุปผู้บริหาร (Executive Summary)

ระบบที่พัฒนาจริง ทำงานได้และมีคุณภาพวิศวกรรมสูงกว่าที่เล่มบรรยายไว้มาก — มี pipeline ครบ (scrape → คัดไทย → clean → tokenize → sentiment → aspect → phrase extraction → insights → dashboard), มีชุดทดสอบ 28 ไฟล์/~135 เทสต์, มีโมดูลประเมินผลเชิงปริมาณ และ WangchanBERTa ใช้งานได้จริง (รายงานผล 88.3% accuracy)

ปัญหาหลักไม่ใช่โค้ดบกพร่อง แต่คือ “เล่มวิจัยกับโค้ดเล่าคนละเรื่องกันบางส่วน”:

- [Critical]** เล่มอ้างว่า **fine-tune WangchanBERTa** ด้วยข้อมูลรีวิวร้านอาหาร (หน้า 23) — โค้ดไม่มีการ fine-tune ใดๆ ใช้ checkpoint สำเร็จรูป `finetuned@wisesight_sentiment` แบบ off-the-shelf
- [Critical]** หัวใจของระบบจริงคือ การสกัด **opinion phrase** เชิงพจนานุกรม + **Gemini LLM** + การจัดหมวด **4 ชั้น + insights** — ทั้งหมดนี้ไม่ปรากฏในเล่ม
- [Critical]** เล่มไม่เอ่ยถึง **Gemini/LLM** แม้แต่คำเดียว ทั้งที่เป็น 1 ใน 2 extraction engine
- [Important]** ค่าเริ่มต้นของระบบคือ **lexicon** ไม่ใช่ **WangchanBERTa** (`USE_MODEL=0`, `extract_engine="rule"`) ขัดกับการที่เล่มชู WangchanBERTa เป็นพระเอก
- [Critical]** บทที่ 4 (ผลการวิจัย) และบทที่ 5 ยังเป็น **template** เปล่า ทั้งที่ `eval/report.txt` มีผลพร้อมใช้แล้ว
- [Minor]** เล่มระบุจัดเก็บเป็น CSV — โค้ดใช้ **SQLite** (`insightreview.db`) + `export CSV`

สรุป: งานวิศวกรรมแข็งแรง แต่เอกสารวิจัย “ต่ำกว่าความจริง” ในส่วนที่เป็นนวัตกรรมจริง และ “เกินความจริง” ในเรื่อง fine-tuning — ทั้งสองทิศทางคือความเสี่ยงในห้องสอบ

2. คะแนนความสอดคล้อง (Consistency Score)

63 / 100

มิติ	คะแนน	เหตุผล
Data pipeline (scrape→clean→token)	90	ตรงเล่มเกือบทั้งหมด
Sentiment method	55	WangchanBERTa มีจริง แต่ default เป็น lexicon + อ้าง fine-tune ที่ไม่มี
Methodology documentation	35	ส่วนนวัตกรรมจริง (phrase/aspect/LLM/insights) ไม่อยู่ในเล่ม
Results (บทที่ 4)	20	เล่มว่าง ทั้งที่โค้ดผลิตผลได้
Framework / Tech scope	75	Flask/Apify/HTML-CSS ตรง; CSV↔SQLite + JS ไม่ตรง
Architecture quality (โค้ดล้วน)	88	แยกชั้นดี เทสต์เยอะ fallback ครบ

3. ตารางช่องว่าง วิจัย-โค้ด (Research-Code Gap Table)

#	Research Claim (เล่ม)	Actual Code	Match	Severity	Recommendation
---	-----------------------	-------------	-------	----------	----------------

1	Fine-tune WangchanBERTa ด้วยรีวิวร้านอาหาร (น.23)	ไม่มี fine-tune; ใช้ finetuned@wisesight_sentiment สำเร็จรูป	✗	Critical	แก้เป็น “ใช้ pre-finetuned/transfer” หรือ fine-tune จริง
2	WangchanBERTa เป็นวิธีหลัก	Default = lexicon (USE_MODEL=0)	⚠	Critical	ระบุ lexicon=baseline, WangchanBERTa=โหมดวิจัย; รายงานทั้งคู่
3	(ไม่มีในเล่ม)	Gemini LLM extraction engine (llm_extract.py)	✗	Critical	เพิ่ม methodology + cost/privacy/fallback
4	“Keyword Extraction” (รูป 1.1)	Opinion-phrase extraction เชิง ไวยากรณ์ (P1–P7, idiom/MWE, negation)	✗	Critical	เขียนบทที่ 3 ใหม่ — เป็น contribution หลัก
5	จำแนกอารมณ์ 3 คลาส	ตรง + clause-level + aspect-level sentiment	⚠	Important	เพิ่มอธิบาย clause split + ABSA-lite
6	(ไม่มีในเล่ม)	จัดหมวด 4 ชั้น food/service/ambience (route_aspect)	✗	Important	UI mockup แสดงหมวดแล้ว แต่วิธีไม่ถูกอธิบาย
7	(ไม่มีในเล่ม)	Actionable insights rule-based (insights.py)	✗	Important	จุดเชื่อมสู่ objective “decision support”
8	จัดเก็บเป็น CSV (น.9)	SQLite + CSV export	✗	Minor	แก้ไขขอบเขตเป็น SQLite (+CSV export)
9	Python/Flask/HTML/CSS	+ JavaScript	⚠	Minor	เพิ่ม JavaScript ในขอบเขต
10	คัดเฉพาะรีวิวไทย	ตรง — is_thai() threshold 0.2	✓	OK	ระบุ threshold ในเล่ม
11	Character Normalization (น.20)	ตรง — _REPEAT_RE	✓	OK	—
12	Stopwords Removal	ตรง — PyThaiNLP	✓	OK	โมเดลรับ raw text (ถูกต้อง)
13	Apify scraper	ตรงเป๊ะ — compass/google-maps-reviews-scraper	✓	OK	—
14	บทที่ 4 ผลการทดลอง	เปล่า; แต่ report.txt มีผล (Acc 88.3%, MacroF1 0.879, Kappa 0.825)	⚠	Critical	เติมบทที่ 4 จากผลที่มี
15	ชุดข้อมูล = รีวิวจริง	demo 30; eval 60 (balance 20/20/20)	⚠	Important	ระบุที่มา/จำนวน/การ label

4. Architecture Review

URL → scraper(Apify/demo) → preprocess(thai/clean/token/clause) → sentiment(WangchanBERTa | lexicon) → aspect(4-tier) → phrase pipeline(rule | Gemini) → aggregate → insights → SQLite → Flask/Jinja dashboard

ด้าน	คะแนน	หมายเหตุ
Module structure	9/10	แยก core/ ชัด, single source of truth (word_polarity , lexicon.py)
Data flow	8/10	dict สัญญาเดียวไหลทั้ง pipeline; clause-level เป็นจุดเด่น
Pipeline design	9/10	deterministic, อธิบายได้, overlap suppression
Error handling / resilience	9/10	fallback ทุกชั้น (model→lexicon, LLM→rule); รายงานสถานะจริง
Scalability	5/10	โมเดล CPU singleton ช้า; Apify sync; ไม่มี queue/async
Maintainability	9/10	คอมเมนต์ไทยละเอียด, เทสต์เยอะ, config รวมศูนย์
Security	7/10	parameterized SQL, Jinja escape, debug ปิด default; จงใจไม่มี auth/CSRF

โดยรวม **Architecture: 8.5/10** — คุณภาพเกินมาตรฐานปริญญาตรี จุดอ่อนเดียวคือโค้ดตึกกว่าที่เล่มสื่อ

5. NLP / ML Review

ขั้นตอน	สถานะในโค้ด	ตรงบทที่ 3?
Data Collection	Apify, field mapping, dedup	✓
Data Cleaning	URL/emoji/special/ยัดเสียง/lowercase	✓
Preprocessing	newmm + stopwords + negation merge + clause split	⚠ บางส่วนไม่อยู่ในเล่ม
Feature Extraction	lexicon grammar (P1–P7) หรือ Gemini JSON-schema	✗ ไม่อยู่ในเล่ม
Model	WangchanBERTa (opt-in) / lexicon (default)	⚠ default ผิดจากที่ชู
Fine-tuning	ไม่มี	✗ ชัดเล่ม น.23
Evaluation	Acc/Prec/Rec/F1/Macro/Weighted/Confusion/Kappa	⚠ ไม่อยู่ในบท 3, บท 4 วาง
Error Analysis	ไม่มีเชิงระบบ	✗ ควรเพิ่ม

ประเด็น **ML** ที่ต้องระวัง:

- ชุดประเมิน **balance** เทียม **20/20/20** → ตัวเลข 88.3% อาจดูดีเกินสภาพใช้งานจริง
- **Neutral recall = 0.70 (ต่ำสุด)** — กลาง→บวก 4, กลาง→ลบ 2; คลาส neutral อ่อนสุด
- **lexicon ≠ WangchanBERTa** — ถ้าสาริตโหมด default ตัวเลขจะคนละชุดกับ report
- **phrase extraction** ไม่มี **gold set** ราย **phrase** → contribution หลักไม่มีหลักฐานเชิงปริมาณ

6. Business Analyst Perspective

องค์ประกอบ (เล่ม)	เชื่อมโยงกับระบบ?
Problem: รีวิวเยอะ/ไม่มีโครงสร้าง/ภาษาไทยซับซ้อน	✓ ระบบแก้ตรงจุด (clause/negation/phrase)
Objective 1: จำแนกอารมณ์	✓ sentiment 3 คลาส
Objective 2: แดชบอร์ดสนับสนุนการตัดสินใจ	✓ dashboard + insights.py (เกินคาด)
Scope: เก็บ CSV	✗ จริงใช้ SQLite
Outcome: เจ้าของร้านใช้วางกลยุทธ์	✓ insights ราย aspect ตอบ “กลยุทธ์” ในชื่อเรื่อง

ข้อสังเกต: ระบบตอบ objective ได้เกินที่เขียน โดยเฉพาะ insights.py ที่แปลง % เป็นคำแนะนำเชิงปฏิบัติ แต่เล่มกลับไม่ได้เครดิตส่วนนี้ — ขยายของต่ำกว่ามูลค่า

7. ความเสี่ยงเชิงวิชาการ (Academic Risks)

1. อ้าง fine-tune แต่ไม่มีในโค้ด → ขอดู training script แล้วไม่มี = เสียความน่าเชื่อถือทั้งเล่ม
2. บทที่ 4-5 วาง = ไม่ผ่านเกณฑ์เนื้อหา
3. Gemini/LLM ไม่อยู่ในเล่ม → กรรมการเปิดโค้ดเจอ = หาวาปิดบัง
4. default lexicon vs ชู WangchanBERTa → สาธิตสดได้ผลคนละชุด
5. contribution หลัก (phrase) ไม่มี quantitative evaluation
6. ชุดประเมิน balance เทียม → ตัวเลขสวยเกินจริง
7. ไม่มี error analysis / neutral อ่อน
8. CSV↔SQLite, JavaScript ตกหล่นจากขอบเขต

8. คำถามที่กรรมการน่าจะถาม (Committee Questions)

เรื่องโมเดล

1. fine-tune WangchanBERTa ด้วยรีวิวจริงไหม? ขอดู training code/loss curve
2. finetuned@wisesight_sentiment ฝึกจากโซเชียลมีเดีย ทำไมใช้กับรีวิวร้านอาหารได้?
3. ทำไม default เป็น lexicon? ตัวเลขในเล่มมาจากไหน?
4. lexicon กับ WangchanBERTa ผลต่างกันเท่าไร?
5. คลาส neutral recall แค่ 0.70 เพราะอะไร?

เรื่องชุดข้อมูล

6. 60 รีวิว label จากไหน ใคร label วัด inter-annotator agreement ไหม?
7. ทำไม balance 20/20/20 พอดี? สะท้อนรีวิวจริงไหม?
8. 60 ตัวอย่างพอสรุปทางสถิติไหม?

เรื่อง phrase extraction / Gemini

9. "Keyword Extraction" จริง ๆ คือ phrase extraction ใช่ไหม ทำไมไม่อธิบาย?
10. Gemini คืออะไร อยู่ตรงไหนของ methodology?
11. ส่งรีวิวให้ Google มีประเด็น privacy ไหม?
12. ค่าใช้จ่าย Gemini เท่าไร quota หมดทำงานยังไง?
13. phrase extraction วัดความแม่นยำอย่างไร?
14. ทำไมเลือกไวยากรณ์พจนานุกรมแทน POS tagging?

เรื่องระบบ/หมวด/เอกสาร

15. หมวด food/service/ambience จัดด้วยวิธีอะไร (4 ชั้น) ทำไมไม่อยู่ในบท 3?
16. clause splitting ทำไมแบ่งเฉพาะ "แต่"? พลาดเคสไหน?
17. negation รองรับแค่คำซ้ำติดกัน 1 คำ จัดการ "ไม่อร่อยเท่าไร" ยังไง?
18. insights ใช้ threshold 0.65/0.30/5 — ที่มาของเลขเหล่านี้?
19. เล่มว่าเก็บ CSV แต่โค้ดใช้ SQLite — อันไหนถูก?
20. บทที่ 4/5 จะเติมอะไร เมื่อไร?
21. ทำไม Tokenization สะกดผิดเป็น "Takenization" (น.20)?
22. ระบบรองรับกี่ผู้ใช้พร้อมกัน? โมเดล CPU 1 รีวิวใช้เวลาเท่าไร?

9. แผนปรับปรุงจัดลำดับความสำคัญ (Prioritized Action Plan)

● CRITICAL — ต้องแก้ก่อนส่งเล่ม

1. ตัดสินใจเรื่อง **fine-tuning**: ทำจริงบน Colab หรือแก้เล่มให้ตรง = “ใช้ pre-finetuned checkpoint (transfer learning) ไม่ได้ fine-tune เพิ่ม”
2. เดิมบทที่ 4 จาก `eval/report.txt` + screenshot dashboard + ระบุว่ารายงานจากโมเดล WangchanBERTa
3. เพิ่ม **methodology** จริง ในบท 3: phrase extraction, negation, clause split, aspect 4-tier, insights
4. เพิ่มหัวข้อ **Gemini engine** + เปรียบเทียบ rule vs LLM (`scripts/compare_engines.py`) + fallback/cost/privacy
5. เดิมบทที่ 5 (สรุป/ข้อจำกัด/ข้อเสนอแนะ) — ดึงจาก Limitations ใน README ได้

● IMPORTANT — ควรแก้

6. ระบุบทบาท lexicon (baseline) vs WangchanBERTa (วิจัย) ให้ชัด
7. อธิบายที่มา/วิธี label ของ 60 รีวิว + เพิ่มจำนวนถ้าได้
8. เพิ่ม error analysis (neutral) + อภิปราย balance เทียม
9. เพิ่ม quantitative eval ของ phrase/aspect
10. แก้ขอบเขต: CSV→SQLite(+CSV export), เพิ่ม JavaScript

● OPTIONAL — เพิ่มความสมบูรณ์

11. แก้คำสะกด: Takenization→Tokenization, Unstructure→Unstructured, Pre-trainnng→Pre-training, Lowik→Lowphansirikul
12. แก้ field “Error! Reference source not found.” (น.13–15)
13. เพิ่ม sequence/architecture diagram ของ pipeline จริง
14. รายงานเวลาประมวลผลจริง (latency) ต่อ 1 การวิเคราะห์

บรรทัดสุดท้าย: โค้ดพร้อมสอบ — เล่มยังไม่พร้อม งานเร่งด่วนคือ “เขียนเล่มให้ตรงกับระบบที่ทำได้ดีอยู่แล้ว” ถ้าแก้ครบ 5 ข้อ Critical คะแนนความสอดคล้องจะขยับจาก 63 ขึ้นระดับ 90+ ได้ เพราะตัวระบบรองรับเกือบทุกคำกล่าวอ้างอยู่แล้ว ยกเว้นเรื่อง fine-tuning ที่ต้องเลือกระหว่าง “ทำจริง” กับ “แก้คำพูด”